Solo.market doc 2 June

Grogoni Technical Architecture
Specification v1.0

Document Status

Document Title: Grogoni Technical Architecture Specification v1.0 □
Document Type: Engineering Implementation Blueprint
Parent Document: Grogoni Architecture v3.0 □
Runtime: Grogoniclaw□
Primary Platform: Mobile-first PWA for low-end Android smartphones
Secondary Platform: Cloud-assisted backend and optional native Android wrapper
Primary Goal: Define how Grogoni should be engineered, deployed, secured, tested, and scaled.

---

1. Technical Overview

Grogoni is implemented as a mobile-first, offline-capable, AI-assisted business operating system.

The system is divided into five major engineering layers:

Client Layer
↓
Runtime Layer
↓
Business Services Layer
↓
Data Layer
↓
Infrastructure Layer

The main technical requirement is that the product must remain usable on low-end smartphones while still supporting advanced AI, sync, payments, documents, marketplace skills, and integrations.

2. Target Technical Constraints

2.1 Device Constraints

The client must support:

Minimum RAM: 1 GB
Preferred RAM: 2 GB+
Primary OS: Android
Primary delivery: PWA
Initial app shell: under 15 MB
Offline storage: required
Local AI model: optional
Background sync: controlled

2.2 Network Constraints

The system must assume:

Unstable mobile data
Slow uploads
Expensive bundles
Temporary offline periods
Partial sync failure
Payment webhook delays
SMS fallback requirement

2.3 AI Constraints

The system must not assume access to large models.

Supported AI modes:

Mode 1: Rule-based command parser
Mode 2: Tiny local SLM
Mode 3: Small local SLM through llama.cpp
Mode 4: Cloud fallback model

Mode 5: Hybrid local-cloud execution

---

3. Recommended Monorepo Structure

```
grogoni-market/
│
├── apps/
│   ├── web/
│   ├── mobile-shell/
│   └── admin/
│
├── services/
│   ├── api/
│   ├── sync/
│   ├── ai-runtime/
│   ├── document-worker/
│   ├── payment-worker/
│   ├── notification-worker/
│   ├── marketplace/
│   └── webhook-gateway/
│
├── packages/
│   ├── shared-types/
│   ├── business-core/
│   ├── template-core/
│   ├── event-core/
│   ├── sync-core/
│   ├── ai-core/
│   ├── tool-core/
│   ├── security-core/
│   └── ui/
│
├── infra/
│   ├── docker/
│   ├── postgres/
│   ├── redis/
│   ├── nginx/
│   ├── observability/
│   └── deployment/
│
├── docs/
```

```
│   ├── architecture/
│   ├── api/
│   ├── database/
│   ├── runtime/
│   └── marketplace/
│
└── tests/
    ├── unit/
    ├── integration/
    ├── e2e/
    ├── load/
    ├── offline/
    └── security/
```

---

4. Client Architecture

4.1 Primary Client

The first client should be a PWA.

Recommended stack:

Frontend: React / Preact / Solid
State: lightweight store
Offline DB: IndexedDB
Local cache: browser storage
Sync: background queue
UI: mobile-first chat interface

The PWA should be installable from the browser and work without a heavy app store dependency.

4.2 Client Modules

Chat Interface
Local Database
Sync Queue
Permission Manager
Notification Manager

File Upload Manager
Camera Capture
Contact Import
Language Manager
Offline Status Manager
AI Command Parser

4.3 Mobile UI Rule

All features must be reachable from chat.

Structured screens are allowed only as support views:

Product picker
Customer picker
Invoice preview
Payment confirmation
Document preview
Settings
Permissions
Reports

---

5. Backend Architecture

5.1 Backend Services

API Service
Sync Service
AI Runtime Service
Document Worker
Payment Worker
Notification Worker
Marketplace Service
Webhook Gateway
Admin Service

5.2 API Service

Responsibilities:

Authentication
Account management
Business CRUD APIs
Permission checks
Tool execution requests
Audit logging
Session management

5.3 Sync Service

Responsibilities:

Receive client deltas
Resolve conflicts
Apply event logs
Return server changes
Track sync cursors
Handle retry-safe writes

5.4 AI Runtime Service

Responsibilities:

Intent routing
Planning
Tool selection
Context building
Cloud fallback
Memory compression
Verification calls
Response generation

5.5 Document Worker

Responsibilities:

PDF parsing
Spreadsheet import

Word document import
Image extraction
Field mapping
Template matching
Preview generation

5.6 Payment Worker

Responsibilities:

Payment provider integration
Webhook handling
Transaction reconciliation
Invoice payment updates
Payment confirmation
Failed payment retries

---

6. Data Architecture

6.1 Local Data

On device:

IndexedDB
Local sync queue
Cached business records
Cached templates
Recent conversations
Draft invoices
Pending uploads

Local data must be encrypted where possible.

6.2 Cloud Data

Cloud storage:

PostgreSQL: source of truth
Redis: cache and queues
Object storage: documents/images
Event log: audit and sync history

6.3 Database Tables

Core tables:

accounts
users
staff_roles
agents
customers
suppliers
products
inventory_movements
invoices
invoice_items
payments
logistics_records
tax_records
credit_accounts
credit_notes
pricing_rules
vocab_aliases
documents
document_imports
events
sync_cursors
audit_logs
skills
installed_skills
tool_permissions
model_registry
memory_entries
knowledge_entries

---

7. Core Database Schema

7.1 accounts

```
CREATE TABLE accounts (
  id UUID PRIMARY KEY,
  business_name TEXT NOT NULL,
  owner_user_id UUID,
  phone_number TEXT,
  email TEXT,
  country TEXT NOT NULL DEFAULT 'KE',
  base_currency TEXT NOT NULL DEFAULT 'KES',
  language TEXT NOT NULL DEFAULT 'en',
  verification_tier INTEGER NOT NULL DEFAULT 0,
  subscription_plan TEXT NOT NULL DEFAULT 'free',
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

7.2 users

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  account_id UUID REFERENCES accounts(id),
  name TEXT,
  phone TEXT,
  email TEXT,
  role TEXT NOT NULL,
  login_method TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

7.3 products

```
CREATE TABLE products (
  id UUID PRIMARY KEY,
  account_id UUID NOT NULL REFERENCES accounts(id),
  supplier_id UUID,
  code TEXT,
  name TEXT NOT NULL,
  description TEXT,
  quantity NUMERIC NOT NULL DEFAULT 0,
```

```
  reorder_threshold NUMERIC DEFAULT 0,
  reorder_quantity NUMERIC DEFAULT 0,
  buying_price NUMERIC,
  selling_price NUMERIC,
  unit_of_measurement TEXT,
  image_url TEXT,
  packaging TEXT,
  source TEXT,
  status TEXT DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

7.4 invoices

```
CREATE TABLE invoices (
  id UUID PRIMARY KEY,
  account_id UUID NOT NULL REFERENCES accounts(id),
  invoice_number TEXT NOT NULL,
  customer_id UUID,
  supplier_id UUID,
  agent_id UUID,
  invoice_date DATE NOT NULL,
  due_date DATE,
  terms TEXT,
  location TEXT,
  subtotal NUMERIC NOT NULL DEFAULT 0,
  tax_amount NUMERIC NOT NULL DEFAULT 0,
  discount_amount NUMERIC NOT NULL DEFAULT 0,
  total_amount NUMERIC NOT NULL DEFAULT 0,
  payment_status TEXT NOT NULL DEFAULT 'unpaid',
  delivery_status TEXT DEFAULT 'not_required',
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at TIMESTAMP NOT NULL DEFAULT now()
);
```

7.5 events

```
CREATE TABLE events (
  id UUID PRIMARY KEY,
  account_id UUID NOT NULL,
  event_type TEXT NOT NULL,
```

```
  aggregate_type TEXT NOT NULL,
  aggregate_id UUID NOT NULL,
  payload JSONB NOT NULL,
  created_by UUID,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  sync_status TEXT DEFAULT 'pending'
);
```

---

8. Event Architecture

Grogoni should be event-driven.

Every important mutation creates an event.

Examples:

ProductCreated
ProductUpdated
StockAdjusted
CustomerCreated
InvoiceCreated
InvoicePaid
PaymentReceived
DeliveryCreated
DeliveryCompleted
DocumentImported
SkillInstalled
TaxCalculated
CreditLimitExceeded

8.1 Event Structure

```
{
  "id": "uuid",
  "account_id": "uuid",
  "event_type": "InvoiceCreated",
  "aggregate_type": "invoice",
  "aggregate_id": "uuid",
  "payload": {},
  "created_by": "uuid",
```

```
  "created_at": "timestamp",
  "sync_status": "pending"
}
```

8.2 Event Rules

Events are immutable.
Events are account-scoped.
Events are used for sync.
Events are used for audit.
Events are used for marketplace skill triggers.
Events are used for reports.

---

9. Sync Protocol

9.1 Sync Flow

Client Action
↓
Write Local Record
↓
Create Local Event
↓
Add to Sync Queue
↓
Upload Delta
↓
Server Validates
↓
Server Applies
↓
Server Returns Changes
↓
Client Applies Changes

9.2 Sync Request

```
{
```

```
  "account_id": "uuid",
  "device_id": "device-id",
  "last_sync_cursor": "cursor",
  "events": []
}
```

9.3 Sync Response

```
{
  "accepted_events": [],
  "rejected_events": [],
  "server_events": [],
  "new_sync_cursor": "cursor",
  "conflicts": []
}
```

9.4 Conflict Rules

Money conflicts require user confirmation.
Quantity conflicts require user confirmation.
Tax conflicts require user confirmation.
Payment records are never overwritten silently.
Latest confirmed user action wins only for low-risk fields.
Both versions are preserved when unsure.

---

10. API Architecture

10.1 API Style

Initial API:

REST JSON APIs
JWT authentication
Account-scoped routes
Role-based authorization
Idempotency keys for writes

Future API:

GraphQL for dashboards
Streaming API for chat
Webhooks for integrations

10.2 Core Routes

POST /auth/otp/request
POST /auth/otp/verify

GET /accounts/:id
PATCH /accounts/:id

GET /products
POST /products
PATCH /products/:id

GET /customers
POST /customers
PATCH /customers/:id

POST /invoices
GET /invoices/:id
POST /invoices/:id/send

POST /payments/record
POST /sync/push-pull

POST /ai/message
POST /tools/execute
POST /documents/upload
GET /marketplace/skills
POST /marketplace/install

10.3 Idempotency

All write routes should support:

Idempotency-Key header

This prevents duplicate invoices or payments during poor connectivity.

---

11. AI Runtime Integration

11.1 AI Message Flow

User Message
↓
Client sends to /ai/message
↓
Context Builder
↓
Intent Router
↓
Planner
↓
Tool Selection
↓
Permission Check
↓
Tool Execution
↓
Verification
↓
Response Generator
↓
Client receives response

11.2 AI Request Payload

```
{
  "account_id": "uuid",
  "user_id": "uuid",
  "agent_id": "uuid",
  "message": "Create invoice for John",
  "locale": "sw-KE",
  "device_context": {
    "online": true,
    "battery": 52,
```

```
    "ram_class": "low"
  }
}
```

11.3 AI Response Payload

```
{
  "message": "Invoice draft created for John.",
  "actions": [],
  "requires_confirmation": true,
  "confirmation_options": [
    "Save invoice",
    "Edit invoice",
    "Cancel"
  ]
}
```

---

12. Tool Execution Architecture

12.1 Tool Definition

```
{
  "name": "create_invoice",
  "description": "Creates an invoice",
  "required_permissions": ["invoice.write", "customer.read", "product.read"],
  "input_schema": {},
  "output_schema": {},
  "risk_level": "medium"
}
```

12.2 Tool Lifecycle

```
Tool Selected
    ↓
Validate Input
    ↓
Check Permissions
    ↓
```

Check Business Rules
↓
Execute
↓
Verify Output
↓
Emit Event
↓
Return Result

12.3 Risk Levels

Low: read-only or draft action
Medium: creates business record
High: changes money, stock, tax, external messages
Critical: payments, deletion, tax submission

High and critical actions require confirmation.

---

13. Model Registry

13.1 Model Registry Table

model_registry
- id
- name
- provider
- model_type
- location
- file_format
- quantization
- min_ram
- task_types
- cost_policy
- status

13.2 Model Types

tiny_local
small_local
vision
speech
reasoning
extraction
cloud

13.3 Model Routing

The router selects models based on:

Task type
Device capability
Network status
Battery
Latency target
Privacy level
Cost policy
Accuracy requirement

---

14. llama.cpp Integration

14.1 Local Model Runtime

Local models should use GGUF where possible.

Execution path:

PWA / Native Shell
↓
Local Runtime Bridge
↓
llama.cpp
↓
GGUF Model

For pure PWA, full local model execution may be limited. A native Android wrapper may be required for stronger local inference.

14.2 Local Model Use Cases

Command classification
Simple entity extraction
Offline product entry
Offline customer lookup
Basic invoice drafting
Short chat assistance

14.3 Cloud Fallback

Cloud fallback is used for:

Large document processing
Complex reasoning
Long reports
Tax summaries
Marketplace skill verification
Vision-heavy tasks

---

15. Memory Architecture

15.1 Memory Tables

memory_entries
- id
- account_id
- agent_id
- memory_type
- scope
- content
- embedding
- importance
- expires_at
- created_at

15.2 Memory Types

conversation
business
customer
knowledge
skill

15.3 Context Builder

Before every AI call, the Context Builder assembles:

Recent conversation
Current task
Relevant customer
Relevant product
Business preferences
Installed skill settings
Critical rules

Older context is summarized.

---

16. Knowledge Layer

16.1 Knowledge Entries

knowledge_entries
- id
- account_id
- topic
- source_type
- source_id
- summary
- metrics
- created_at
- updated_at

16.2 Knowledge Examples

Top customers
Slow-moving products
Stock risk
Credit risk
Weekly sales trend
Supplier performance
Agent performance

The Knowledge Layer powers strategic answers.

---

17. Document Processing Architecture

17.1 Upload Flow

Upload File
↓
Store Original
↓
Detect Type
↓
Parse
↓
Extract Fields
↓
Map to Template
↓
Generate Preview
↓
User Confirms
↓
Save Records

17.2 Supported Documents

PDF
DOCX
XLSX

CSV
Images
Receipts
Supplier catalogs
Product lists
Invoices

17.3 Import Safety

No extracted data should be saved permanently without preview and confirmation.

---

18. Payment Architecture

18.1 Payment Providers

Initial priority:

M-Pesa
Manual payment recording
SMS payment confirmation parsing

Later:

Stripe
PayPal
Regional mobile money providers

18.2 Payment Flow

Invoice Created
↓
Payment Link Generated
↓
Customer Pays
↓
Webhook Received
↓

Transaction Verified
↓
Invoice Updated
↓
Event Emitted
↓
Customer Balance Updated

18.3 Payment Safety

Never trust client-side payment status.
Verify webhooks.
Store transaction references.
Prevent duplicate payments.
Require confirmation for refunds.

---

19. Notification Architecture

Notification channels:

In-app
Push notification
SMS
WhatsApp
Email

Notification events:

Invoice sent
Payment received
Stock low
Delivery update
Debt reminder
Sync failed
Skill installed
Suspicious action

---

20. SMS and USSD Architecture

20.1 SMS

SMS supports:

Invoice summary
Payment link
Stock alert
Debt reminder
Delivery confirmation
Payment confirmation parsing

20.2 USSD

USSD supports:

Check balance
Confirm delivery
Request invoice
Confirm payment

USSD flows must stay short and stateless.

---

21. Marketplace Technical Architecture

21.1 Skill Package

skill.json
tool_definitions.json
permissions.json
templates/
workflows/
tests/
README.md
signature

21.2 Skill Installation Flow

User selects skill
↓
Show permissions
↓
User confirms
↓
Verify signature
↓
Install metadata
↓
Register tools/templates
↓
Emit SkillInstalled event

21.3 Skill Sandbox

Skills cannot:

Access raw database directly
Access tokens
Bypass permissions
Execute hidden network calls
Modify payment records without tool approval

---

22. Security Architecture

22.1 Authentication

Phone OTP
Email OTP
OAuth for linked services
JWT sessions
Refresh tokens
Device registration

22.2 Authorization

Role-based access control
Permission-scoped tools
Account-scoped data
Skill permissions
Sensitive-action confirmation

22.3 Encryption

TLS in transit
Database encryption at rest
Encrypted secrets
Optional local encrypted storage
Token vault for OAuth credentials

22.4 Audit Logs

Log:

Who acted
What changed
When it happened
Which device
Which agent
Which tool
Before/after values for sensitive records

---

23. Compliance Architecture

23.1 KYC/KYB

Verification tiers:

Tier 0: phone only
Tier 1: ID document
Tier 2: business registration
Tier 3: full KYB

23.2 Tax

Tax engine must support configurable rules:

country
tax_type
rate
product_category
effective_date
exemptions

23.3 Data Residency

Each account should store:

home_country
allowed_regions
consent_status
data_export_status

---

24. Observability

24.1 Logs

Capture:

API requests
Sync attempts
AI requests
Tool calls
Payment webhooks
Document processing jobs
Skill installations
Security events

24.2 Metrics

Track:

Latency
Error rate
Sync success rate
AI fallback rate
Tool failure rate
Payment reconciliation time
Document import success rate
Offline usage rate

24.3 Tracing

Trace across:

Client
API
AI Runtime
Tool Executor
Database
Queue
External API

---

25. Testing Strategy

25.1 Unit Tests

Test:

Business rules
Pricing rules
Tax rules
Sync merge logic
Tool validation
Intent parsing helpers

25.2 Integration Tests

Test:

Invoice creation
Payment webhook
Document import
Sync push-pull
Skill installation
Permission enforcement

25.3 Offline Tests

Test:

Create invoice offline
Record payment offline
Reconnect and sync
Conflict resolution
Duplicate prevention

25.4 Device Tests

Test on:

1 GB RAM Android phone
2 GB RAM Android phone
Weak network
Offline mode
Low battery
Small screen

25.5 AI Evaluation

Evaluate:

Intent accuracy
Entity extraction accuracy
Clarification behavior
Tool selection accuracy

Unsafe-action blocking
Local vs cloud fallback behavior

---

26. Deployment Architecture

26.1 Development

Local Docker Compose
PostgreSQL
Redis
API service
Web app
Worker services

26.2 Staging

Cloud PostgreSQL
Cloud Redis
Object storage
CI/CD deployment
Test payment credentials
Test SMS provider

26.3 Production

API cluster
Worker cluster
Managed PostgreSQL
Managed Redis
Object storage
CDN
Monitoring
Backup system
Secrets manager

---

27. Backup and Disaster Recovery

Backups:

Daily PostgreSQL backup
Object storage versioning
Event log retention
Encrypted backup storage
Periodic restore tests

Recovery goals:

Restore business records
Restore invoices
Restore payments
Restore documents
Restore audit trails

---

28. Performance Targets

Chat screen load: under 3 seconds on low-end phone
Basic local action: under 500 ms
Invoice creation: under 2 seconds local/draft
Sync cycle: under 30 seconds for small deltas
API p95 latency: under 500 ms for core routes
Document import: async
Payment update: webhook-driven

---

29. Release Milestones

Milestone 1: Local Business Records

PWA
OTP login
Products
Customers

Invoices
Local storage
Basic chat parser

Milestone 2: Sync

Cloud account
Push-pull sync
Conflict handling
Audit events

Milestone 3: Payments

Manual payments
M-Pesa integration
Payment status
Debt tracking

Milestone 4: Documents

Spreadsheet import
PDF import
Preview confirmation
Template mapping

Milestone 5: AI Runtime

Intent Router
Planner
Tool Executor
Memory
Cloud fallback

Milestone 6: Marketplace

Skill registry
Skill install
Skill permissions
Skill sandbox

30. Final Technical Position

Grogoni should be engineered as:

A mobile-first offline-capable PWA
+
A deterministic business runtime
+
A tool-governed AI runtime
+
An event-driven sync system
+
A cloud-assisted model and document platform
+
A sandboxed marketplace

The most important engineering rule is:

AI should assist the business runtime, not replace it.

Business-critical logic must remain deterministic, auditable, permissioned, and recoverable.

Grogoniclaw provides intelligence, planning, routing, memory, and orchestration.
Grogoni provides the business operating system, data ownership, sync, compliance, marketplace, and user experience.